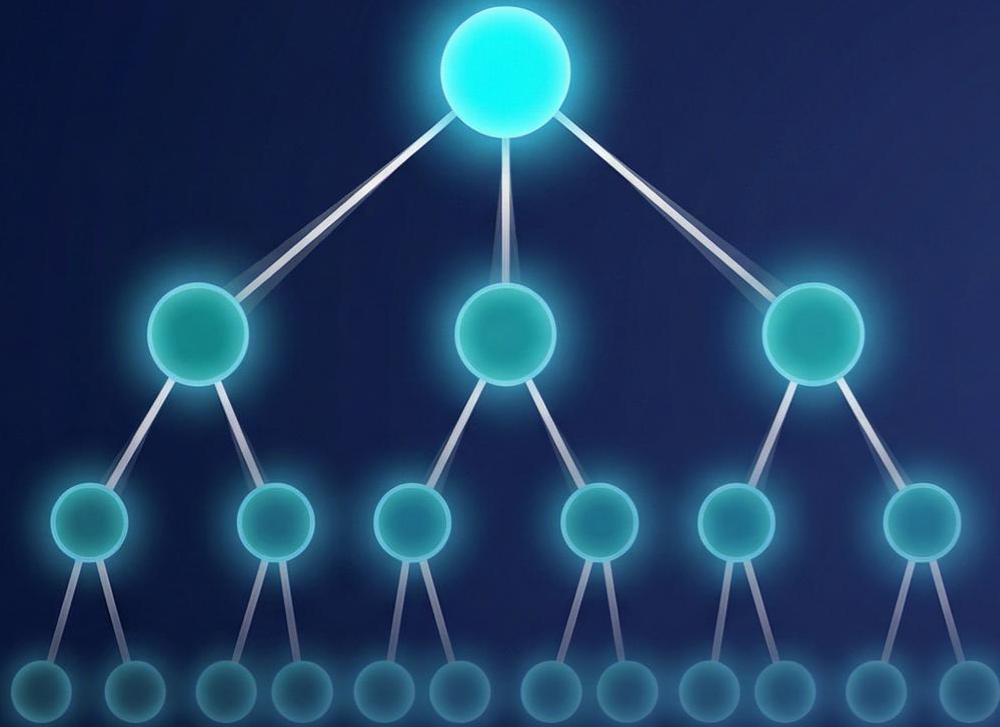




AE1FAI / COMP1037 — Session 2

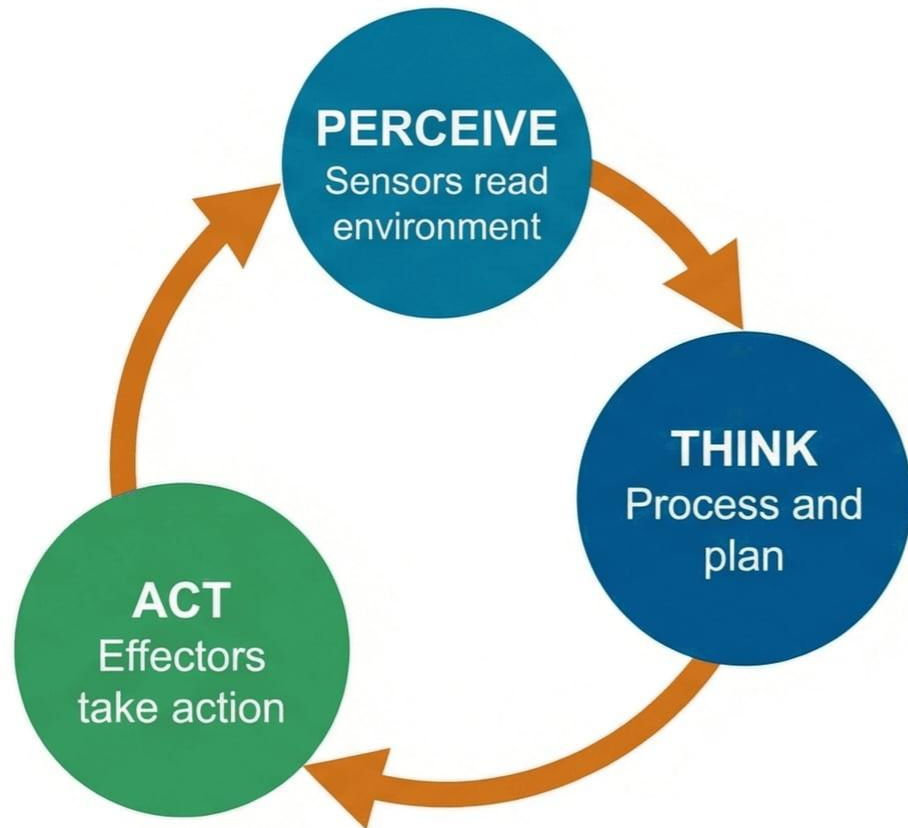
Problem Formulation and Search Trees

Giving the Agent a World Model

Fundamentals of AI (FAI)
Spring | UNNC

Session 1 Recap: The Agent Loop

The Agent Loop



Every intelligent agent lives in this loop

Today's Question

When the agent **THINKS...**
what is it thinking **ABOUT?**

It needs a structured representation
of the world → Problem Formulation

Session 2 → World Model (TODAY)

Session 3 → Search Algorithms

Session 4–5 → Heuristic Search

This Session's Mission: A World Model

SESSION OBJECTIVE

Give our Agent a **WORLD MODEL**

Problem Formulation



States

All possible situations the agent might be in

(Arad), (Bucharest), (0,0,9)



Actions

What the agent can do to change state

Drive, Fill, Pour, Slide



Goals

What success looks like

Reach Bucharest, Get 7L

Without a world model, the agent is blind. With one, it can **PLAN**.

Agent-centric framing

Learning Objectives

1

Define problems using 4 components

Initial State · Actions · Goal Test · Path Cost

2

Apply formulation to classic problems

Romania Map · 8-Puzzle · 8-Queens · Water Jug

3

Understand the Search Tree

Nodes = States · Edges = Actions · depth d · choices per step

4

See why Action definition controls search size

Smarter actions → fewer choices → feasible search

Today's Journey

- [DONE] Recap: Agent Loop
- [NOW] Part 1: Why Formulation Matters**
- [NEXT] Part 2: 4 Components + Examples
- [NEXT] Part 3: Water Jug Case Study
- [NEXT] Part 4: Search Tree and Complexity

PART 2 OF 5

Why Formulation Matters

The Art of Abstraction

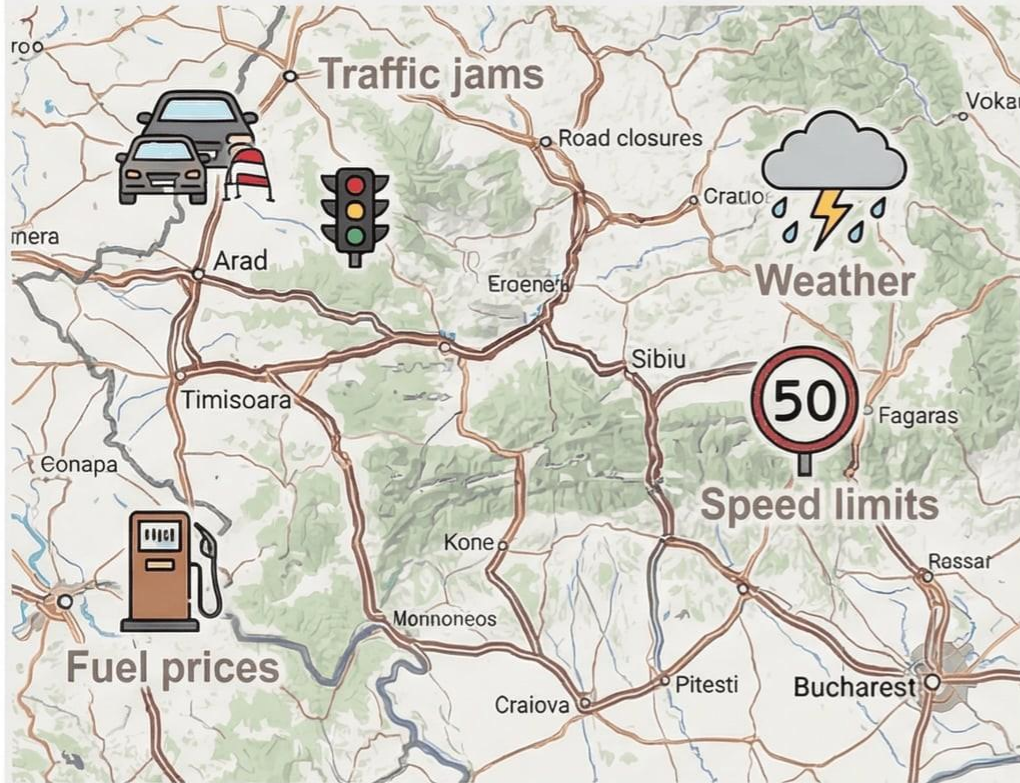
In this section:

- Real world chaos vs. clean model
- The 4 components framework
- Romania Map, 8-Puzzle, 8-Queens examples



Why Formulation Matters: Abstraction

Real World

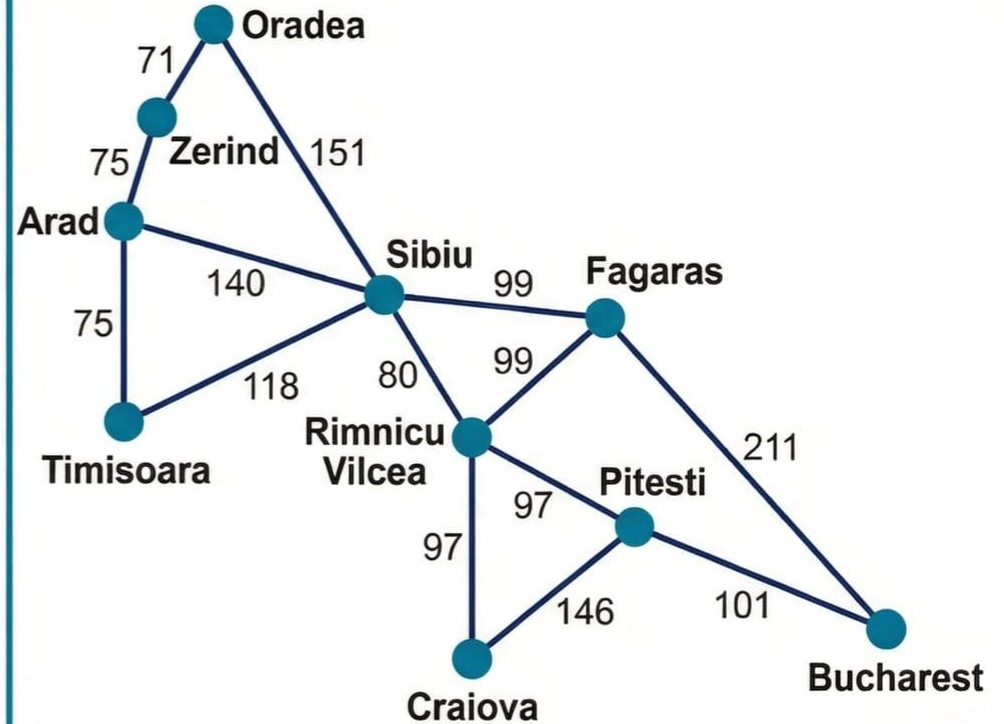


Too complex for direct computation



ABSTRACTION

Agent's Model



Tractable for search algorithms

Key Insight: We keep only what matters for the task. Everything else is discarded.

The 4 Components of a Problem

1 Initial State

Where does the agent START?

- e.g., Agent is in Arad
- e.g., Board = [2,8,3,1,6,4,7,_,5]

2 Actions

What can the agent DO?

- e.g., Drive to adjacent city
- e.g., Slide blank tile UP/DOWN/LEFT/RIGHT

3 Goal Test

How does the agent know it SUCCEEDED?

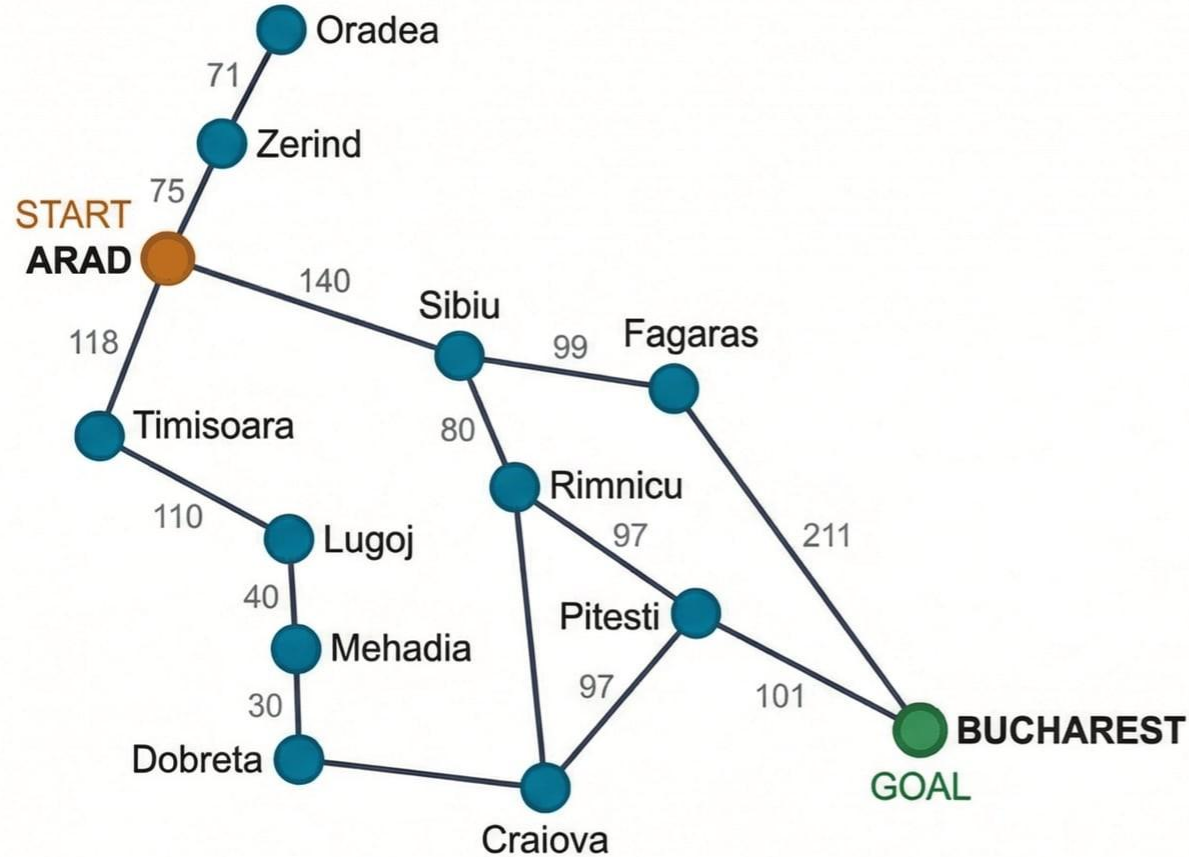
- e.g., `currentCity == Bucharest?`
- e.g., 7 in (jA, jB, jC)?

4 Path Cost

How do we measure solution QUALITY?

- e.g., Total km driven
- e.g., Number of moves

Example 1: Romania Map



Romania Road Network

Applying the 4 Components

1. Initial State Agent is in Arad

```
state = 'Arad'
```

2. Actions Drive to any directly connected city

```
actions('Arad') = ['Zerind', 'Sibiu', 'Timisoara']
```

3. Goal Test Is current city Bucharest?

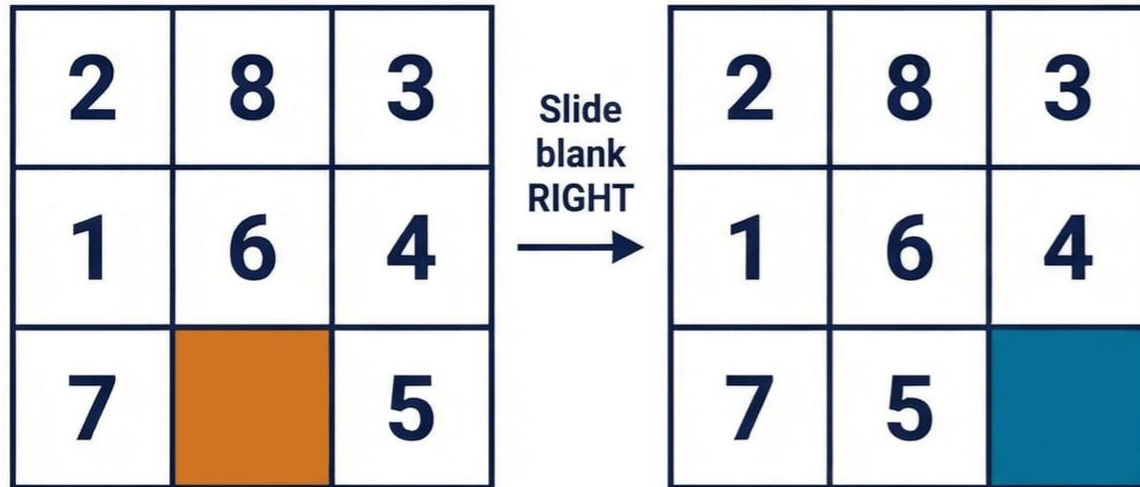
```
goal_test(state) = (state == 'Bucharest')
```

4. Path Cost Total kilometers driven

```
cost = sum of edge weights
```

Example 2: 8-Puzzle — How Action Choice Changes Search Size

The 8-Puzzle



GRID 1
Current State

GRID 2
New State

Initial State: any scrambled configuration

Goal State: [1,2,3,4,5,6,7,8,_]

Path Cost: number of moves

How Action Definition Changes Search Size

Approach A: Move any tile into the blank

Actions: slide any of 2-4 adjacent tiles

Each state has more choices to consider

$9! / 2 = 181,440$ reachable states

With solution depth ~ 20 : enormous search tree

**More choices
per step**

Approach B: Move the BLANK tile (Up/Down/Left/Right)

Actions: UP, DOWN, LEFT, RIGHT (if valid)

Fewer choices per step (at most 4)

Physically identical moves, cleaner model

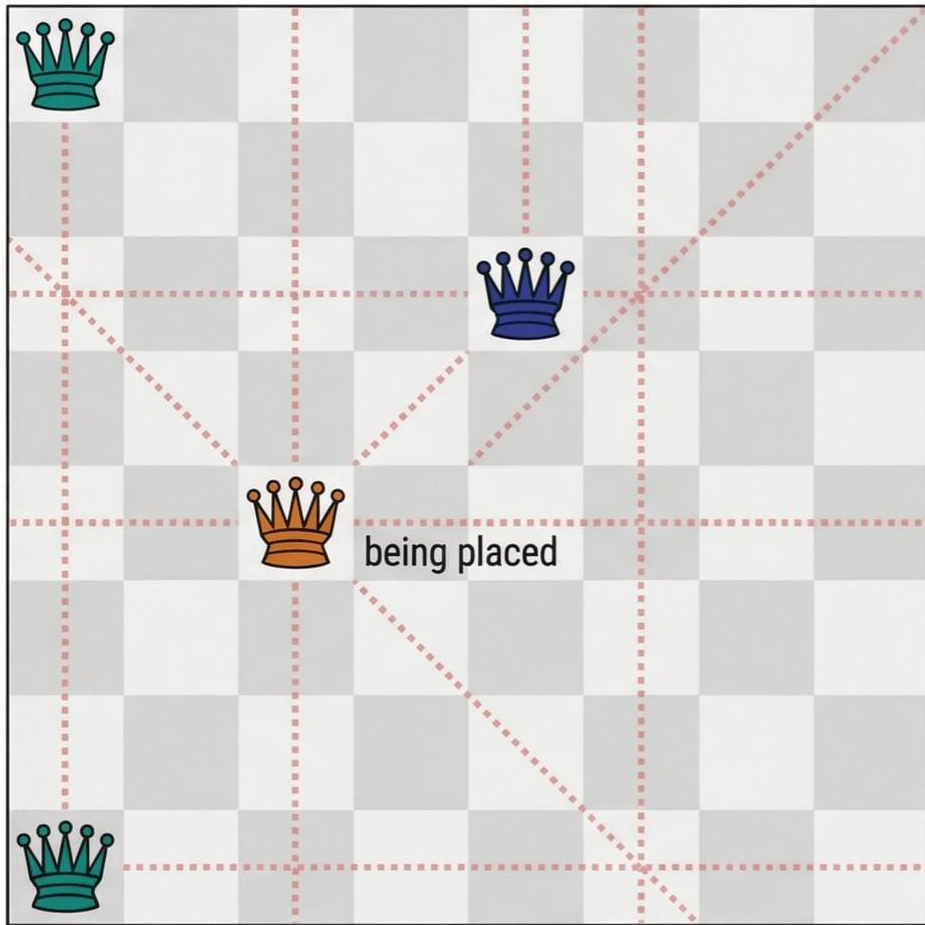
Much smaller search tree!

**Fewer choices
per step**

Key Insight: Same physical puzzle — but 'move blank' gives fewer choices per step than 'move tile'. How you define actions determines how large the search tree grows.

Example 3: 8-Queens — Smarter Actions Mean Fewer Choices

The 8-Queens Problem



Goal: 8 queens, none attacking each other

92 solutions exist

Three Ways to Define Actions

Approach 1: Place queen on any empty square

Actions: choose any of the 64, 63, 62... empty squares

Choices at each step: $64 \times 63 \times \dots \times 57$

Total possibilities: approximately 1.8×10 to the power of 14

INTRACTABLE

Approach 2: Place in leftmost empty column

Actions: choose any row in the next empty column

Choices at each step: 8 (one per row)

Total possibilities: 8 to the power of 8 = 16,777,216

**Better, but
still large**

Approach 3: Only non-attacked squares in next column

Actions: place only in squares not under attack

Choices per step: roughly 1 to 6 (average about 5)

Finds all 92 solutions efficiently!

TRACTABLE

Formulation Controls Search Size: The Core Lesson

Problem	Action Definition	Choices Per Step	Total Search Space
8-Puzzle (smart)	Move blank tile (Up/Down/Left/Right)	at most 4	~3.5 billion (depth 20)
8-Queens (naive)	Place queen anywhere on board	up to 64	1.8×10^{14}
8-Queens (smart)	Non-attacked squares, next column only	about 5	~390,625
Water Jug (3 jugs)	Fill / Empty / Pour between jugs	at most 9	manageable
Chess	Any legal move	about 35	10^{123} (!)

The Key Principle

How you define Actions determines how many choices exist at each step. Fewer choices per step = smaller search tree = feasible search.

This is why formulation IS the algorithm.

Think-Pair-Share

For 8-Queens: can you think of another way to reduce choices further?

Hint: think about board symmetry...

(30 seconds, discuss with neighbor)

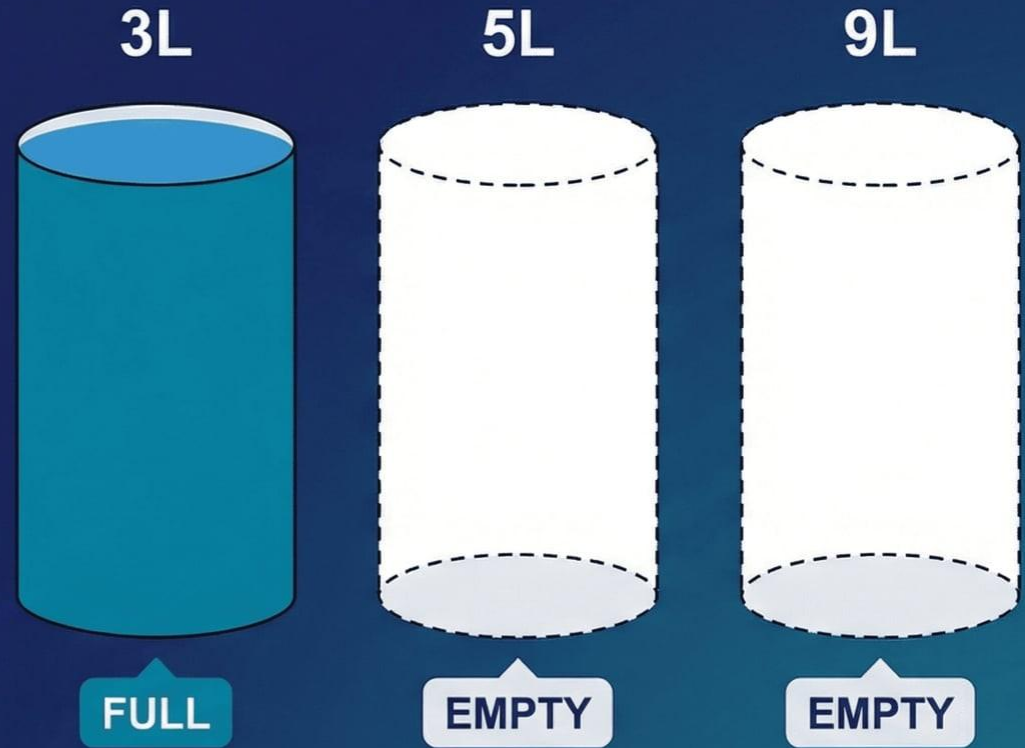
PART 3 OF 5

Interactive Case Study

The Water Jug Problem

HANDS-ON ACTIVITY

- Build the formulation step by step
- Discover states, actions, solutions together
- See the search tree emerge

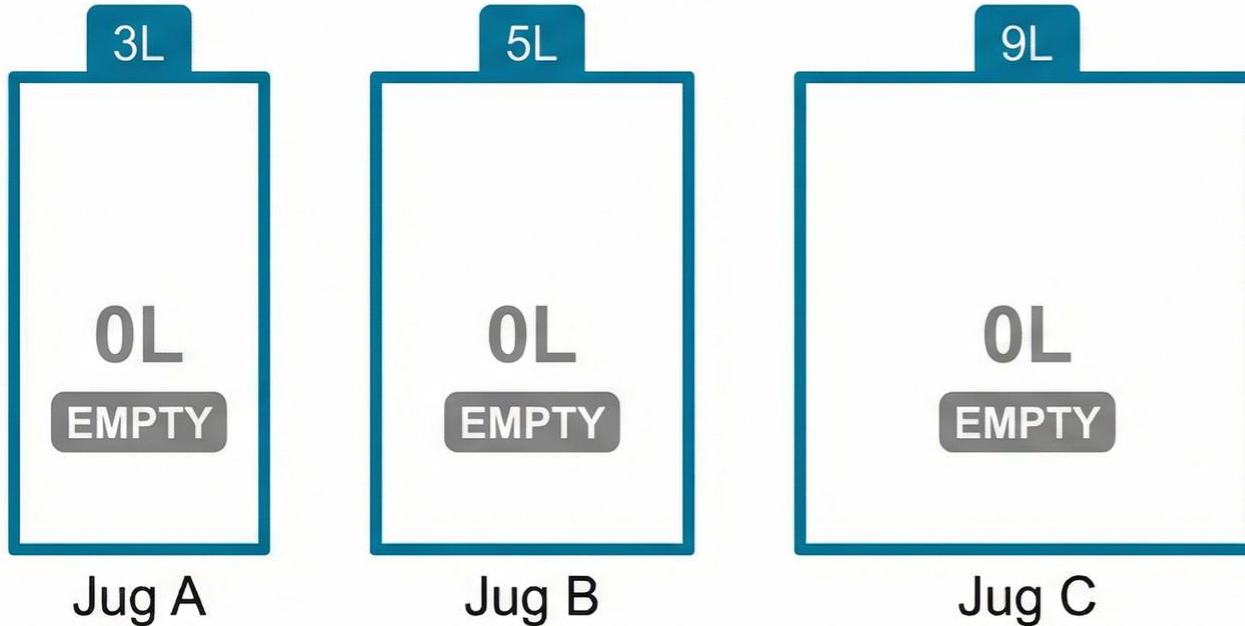


→ ?

Goal: Get exactly 7L in any jug

The Challenge: Measure Exactly 7 Litres

The Setup



Initial State: (0, 0, 0)

Format: (Jug A, Jug B, Jug C)

Water source: unlimited external tap

The Goal

7L
in any one jug
`goal_test(s) = (7 in s)`

State Space Analysis

Jug A: 0 to 3 → 4 possible values

Jug B: 0 to 5 → 6 possible values

Jug C: 0 to 9 → 10 possible values

Total states: $4 \times 6 \times 10 = 240$ (all reachable!)

What is your first move?

(0, 0, 0) → ?

Water Jug Problem setup — no conservation constraint

Step 1: What is a State?

Formal Definition

A **state** captures **ALL** information the agent needs to decide its next action.
(Current state is sufficient — no history needed)

For the Water Jug Problem:

state = (a, b, c)
where:
 a in {0, 1, 2, 3} (3L jug)
 b in {0, 1, 2, 3, 4, 5} (5L jug)
 c in {0, 1, ..., 9} (9L jug)
Initial State: (0, 0, 0)
State space: $4 \times 6 \times 10 = 240$ states

What we do NOT include:

- ✗ How we got here (history)
- ✗ Which moves we made before
- ✗ Time elapsed

These are irrelevant to the next decision

State Examples



(0, 0, 0) — Initial State



(3, 0, 6) — After Fill A, Pour C



(2, 0, 7) — GOAL!
7 in Jug C



(3, 5, 9) — VALID
All jugs filled from tap — allowed!

Step 2: What is an Action?

Types of Actions

↑ FILL — Fill a jug from the tap

Fill(A): $(a,b,c) \rightarrow (3, b, c)$ [tap fills A to capacity]

Fill(B): $(a,b,c) \rightarrow (a, 5, c)$ [tap fills B to capacity]

Fill(C): $(a,b,c) \rightarrow (a, b, 9)$ [tap fills C to capacity]

(Unlimited external water source)

↓ EMPTY — Empty a jug to the drain

Empty(A): $(a,b,c) \rightarrow (0, b, c)$ [drain A completely]

Empty(B): $(a,b,c) \rightarrow (a, 0, c)$ [drain B completely]

Empty(C): $(a,b,c) \rightarrow (a, b, 0)$ [drain C completely]

(Water discarded to sink)

→ POUR — Pour from jug i to jug j

Until: jug i is empty OR jug j is full

Pour(A→B), Pour(B→A)

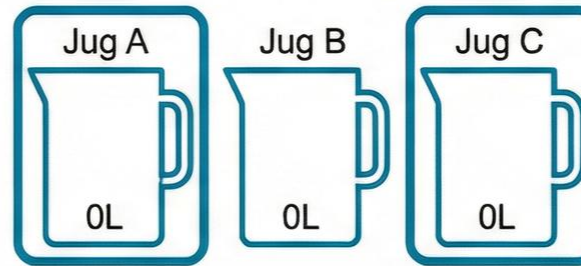
Pour(A→C), Pour(C→A)

Pour(B→C), Pour(C→B)

6 pour directions total

Action Example: Fill(C) from Tap

BEFORE state: (0, 0, 0)

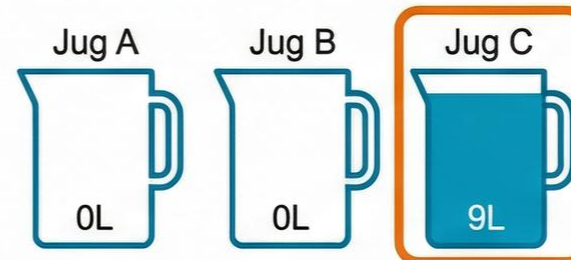


Fill(C)



Fill Jug C from external tap (capacity 9L)

AFTER state: (0, 0, 9)



How many valid actions per state?

3 Fill + 3 Empty + 6 Pour = 12 total

But many are invalid:

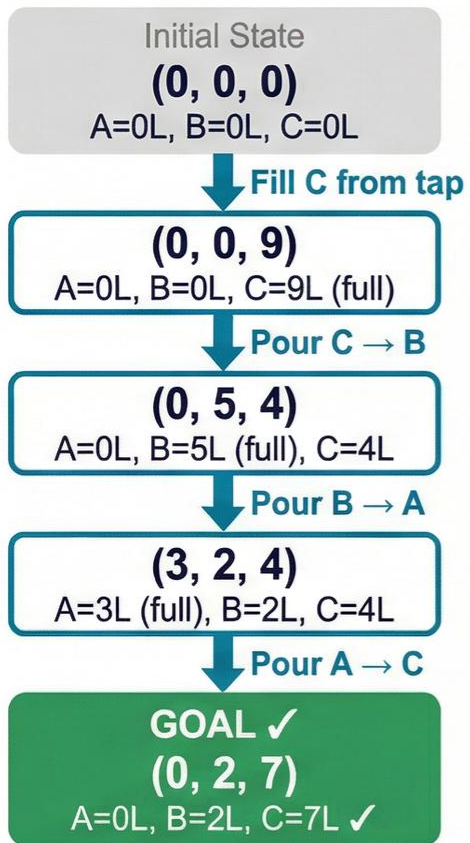
- Cannot fill an already-full jug
- Cannot empty an already-empty jug
- Cannot pour between two empty jugs

Valid actions per state: typically 6 to 9

Water Jug Problem (3L, 5L, 9L) — Goal: measure exactly 7L in any jug | Initial State: (0, 0, 0)

SOLUTION A

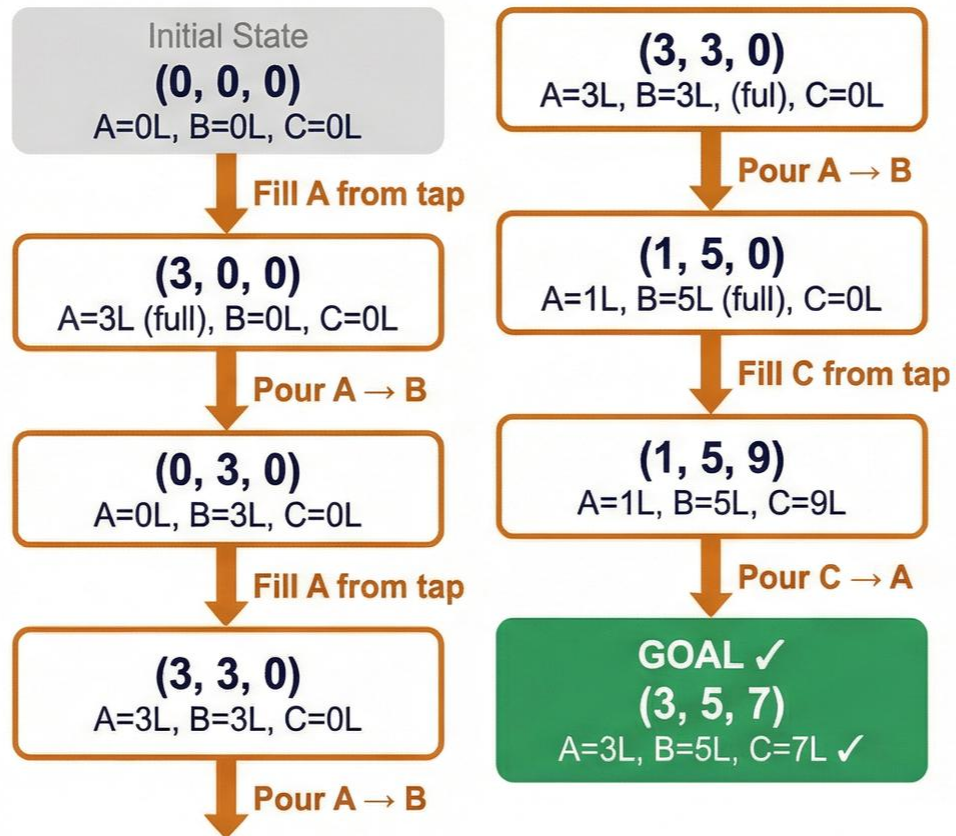
4 actions — shorter path



Path Cost = 4 steps

SOLUTION B

6 actions — longer path



Path Cost = 6 steps

What is a Solution?

A solution is a sequence of actions that transforms the initial state into a goal state.

Path Cost Compares Solutions

A Cost = 4 **B** Cost = 6

Both are valid solutions!

! Is A Optimal?

We know A costs less than B. But to confirm A is OPTIMAL, we must browse ALL possible solution paths and verify none has fewer than 4 steps.

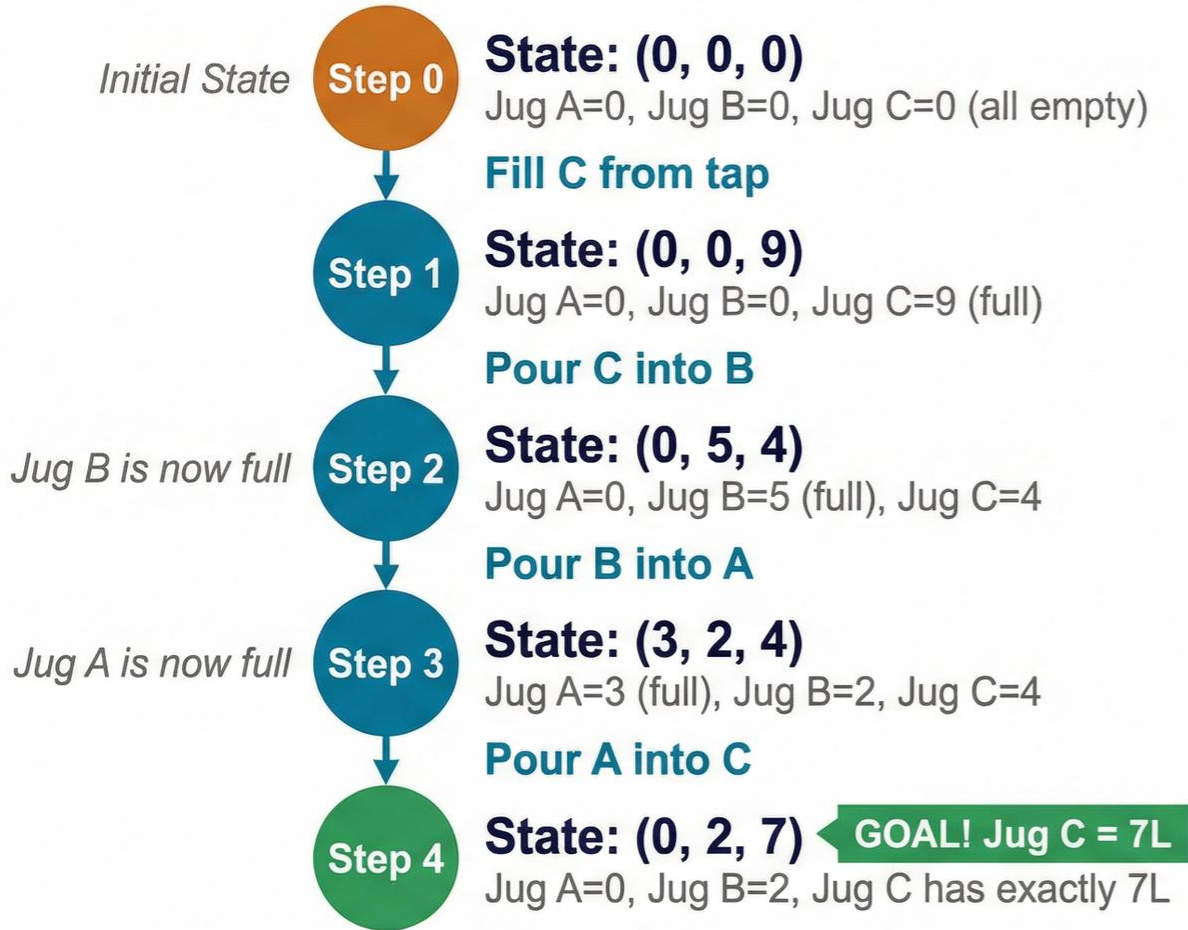
This requires exhaustive search.

AIMA Ch.3.1 — Problem Definition

Solution = sequence of actions from initial state to goal state

Step 3: Solution and Path Cost

A Solution Path



Solution found in 4 steps!

Starting from empty jugs (0,0,0).

Path Cost and Optimality

Path Cost = number of actions taken

Each action costs 1 step

Optimal solution = fewest steps

Solution A: 4 steps (shown above)

$(0,0,0) \rightarrow (0,0,9) \rightarrow (0,5,4) \rightarrow (3,2,4) \rightarrow (0,2,7)$

Lower cost than Solution B

But is it optimal? We need to check ALL paths!

Solution B: 6 steps (another route)

Takes a longer path to the same goal

Still correct, but higher cost

Session2_Demo_Problem_Formulation_BFS.ipynb

```
def action_cost(problem, state, action):  
    return 1 # uniform cost
```

```
# Path cost = sum of action costs  
# cost(path) = len(path) # 1 per step
```

Direct translation of the 4-component framework

The Full Formulation

Formal Definition

1. Initial State

$s_0 = (0, 0, 9)$

(Jug A=0, Jug B=0, Jug C=9 — C is full)

2. Actions

Fill(X), Empty(X), Pour(X to Y)

for all jugs X, Y in {A, B, C}, X not equal to Y

Typically 6 to 9 valid actions per state

3. Goal Test

$\text{goal_test}(s) = (7 \text{ in } s)$

(True if any jug contains exactly 7L)

4. Path Cost

$\text{cost}(\text{path}) = \text{len}(\text{path})$

(Each action costs 1 step)

Python Implementation

Problem_formulation_lecture_demo.ipynb

```
class WaterJugProblem:
    def __init__(self):
        # Initial State: C is full (9L)
        self.initial = (0, 0, 9)
        self.caps = (3, 5, 9)

    def actions(self, state):
        """Return valid actions from state"""
        acts = []
        a, b, c = state
        ca, cb, cc = self.caps
        # Fill actions
        if a < ca: acts.append('Fill A')
        if b < cb: acts.append('Fill B')
        # ... (more actions)
        return acts

    def goal_test(self, state):
        """Check if goal reached"""
        return 7 in state

    def path_cost(self, path):
        return len(path)
```

Direct translation of the 4-component framework

PART 4 OF 5

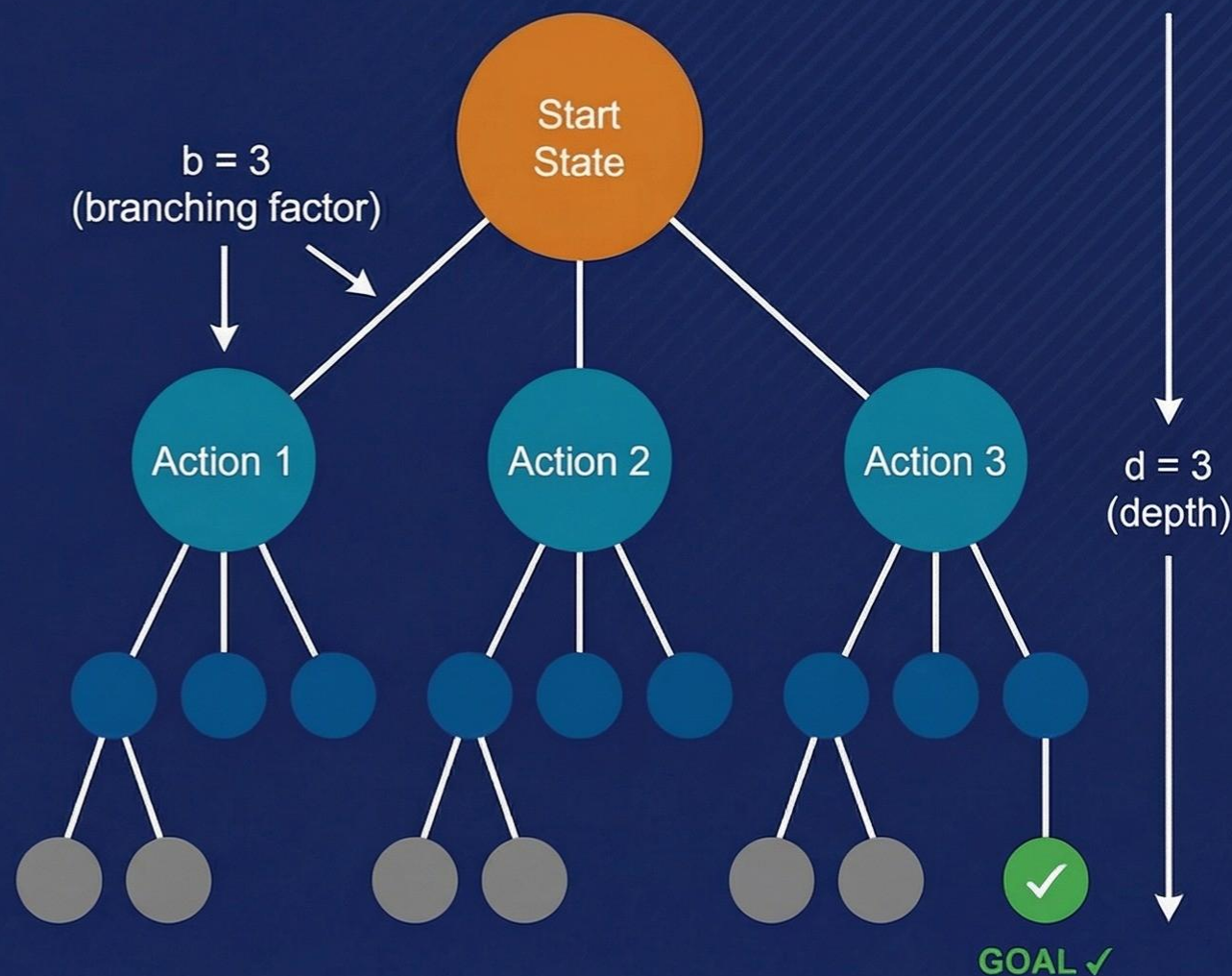
The Agent's Perspective

The Search Tree



FROM FORMULATION TO COMPUTATION

- How formulation creates the search tree
- Branching factor b and depth d
- Why b^d determines everything



How Formulation Defines the Tree

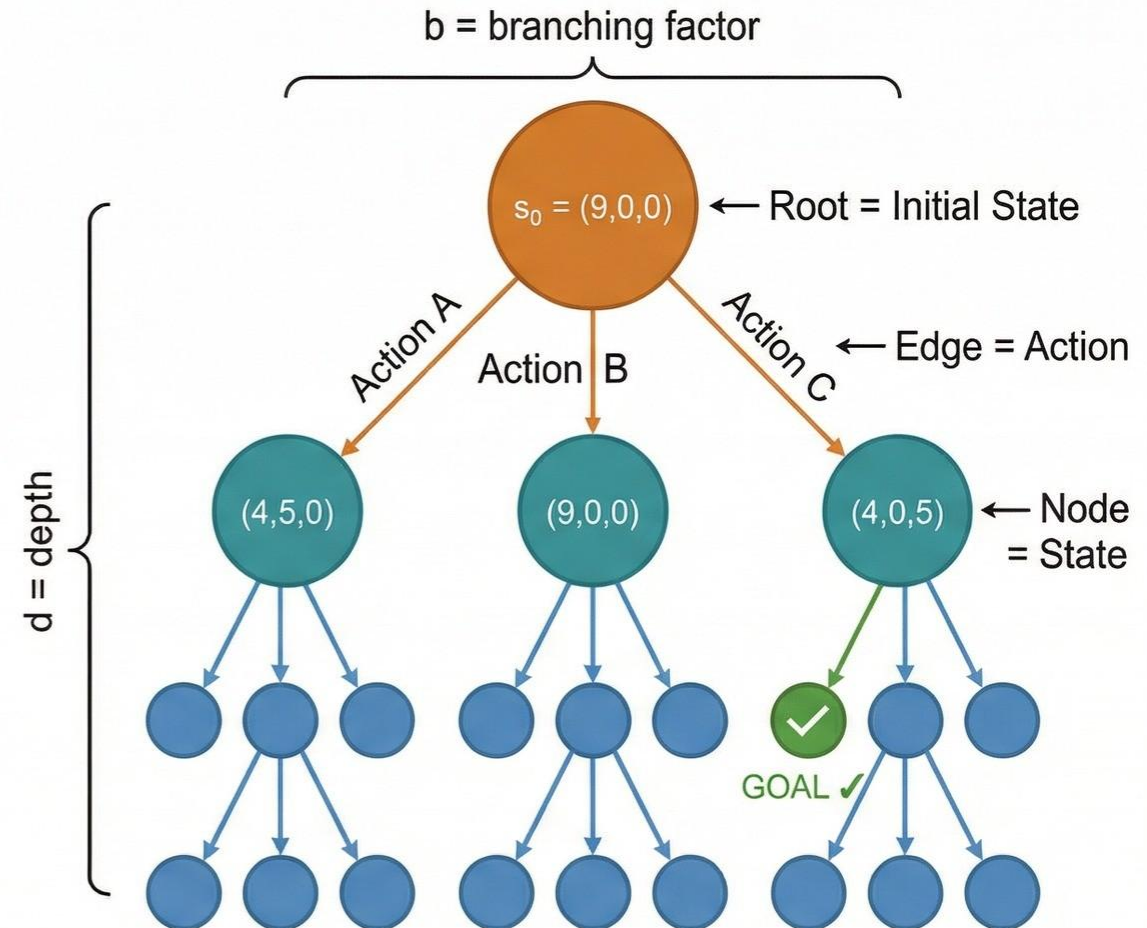
Problem Formulation → Search Tree

	Formulation	Search Tree
1	 Initial State	 Root Node
2	 Actions	 Edges / Branches
3	Transition Model	How children are generated
4	 Goal Test	 Goal Nodes
5	 Path Cost	Edge weights / path length

b = branching factor = avg valid actions per state

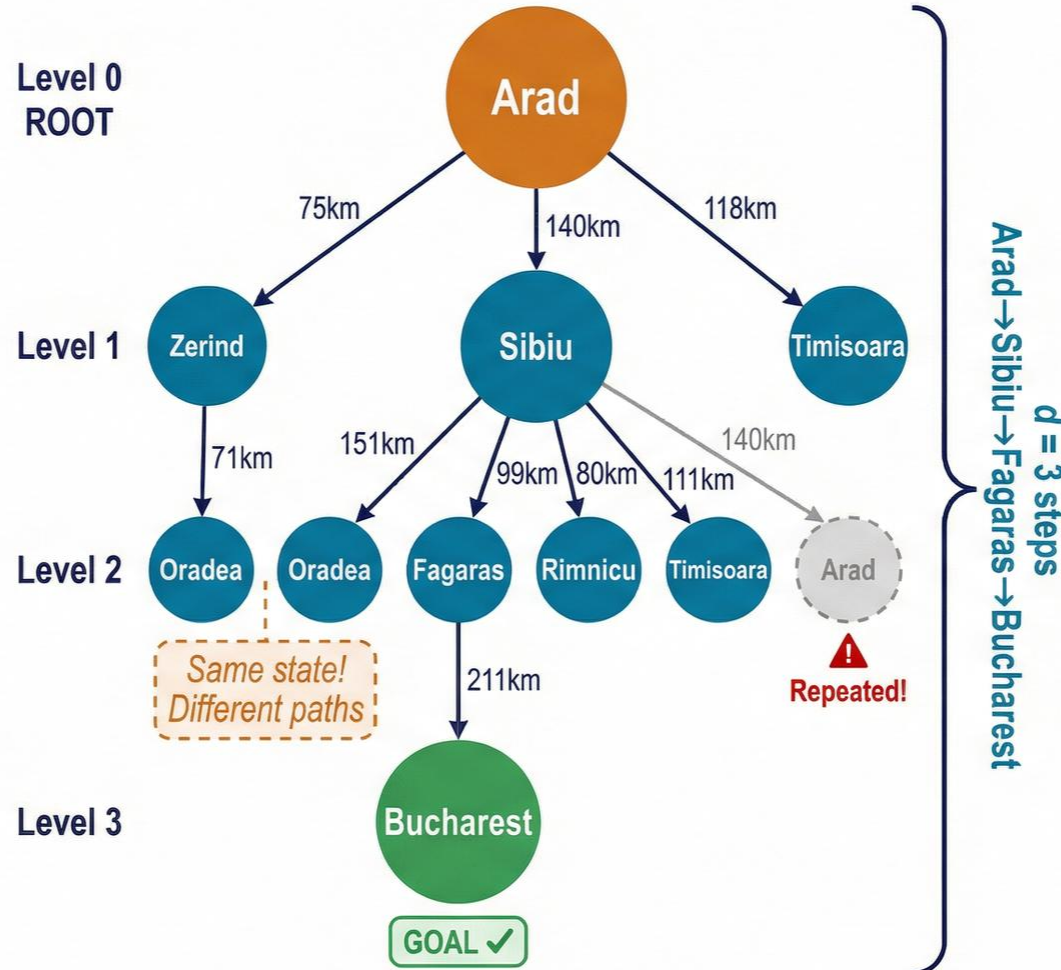
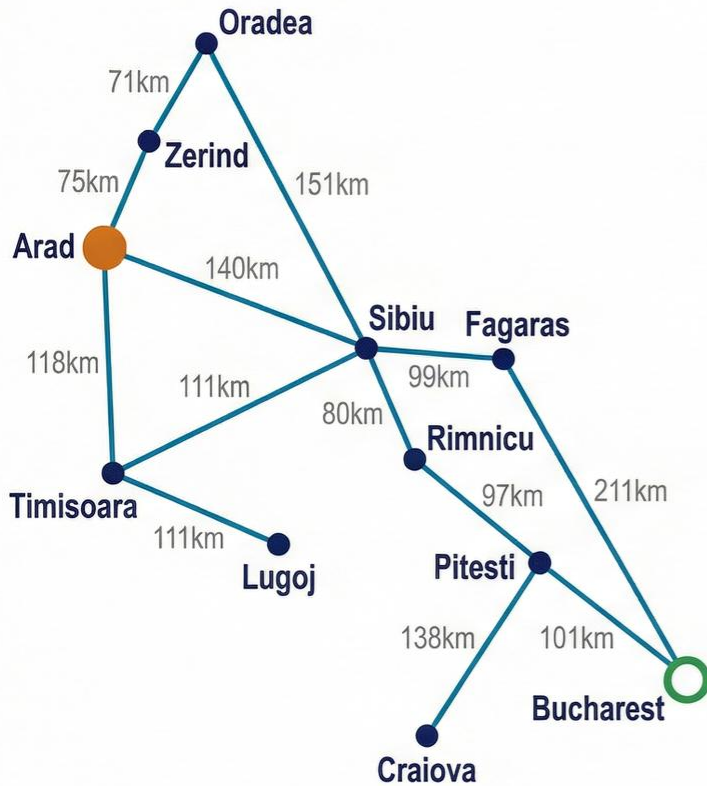
d = depth = length of solution path

Total nodes explored $\approx b^d$



Visualizing the Search Tree (Romania Map)

Romania Road Map



Key Observations

Repeated States Problem

- Arad appears as child of Sibiu
- Sibiu came FROM Arad — repeated state
- Can loop infinitely in cyclic spaces
- **Must track visited states**

Multiple Paths to Same State

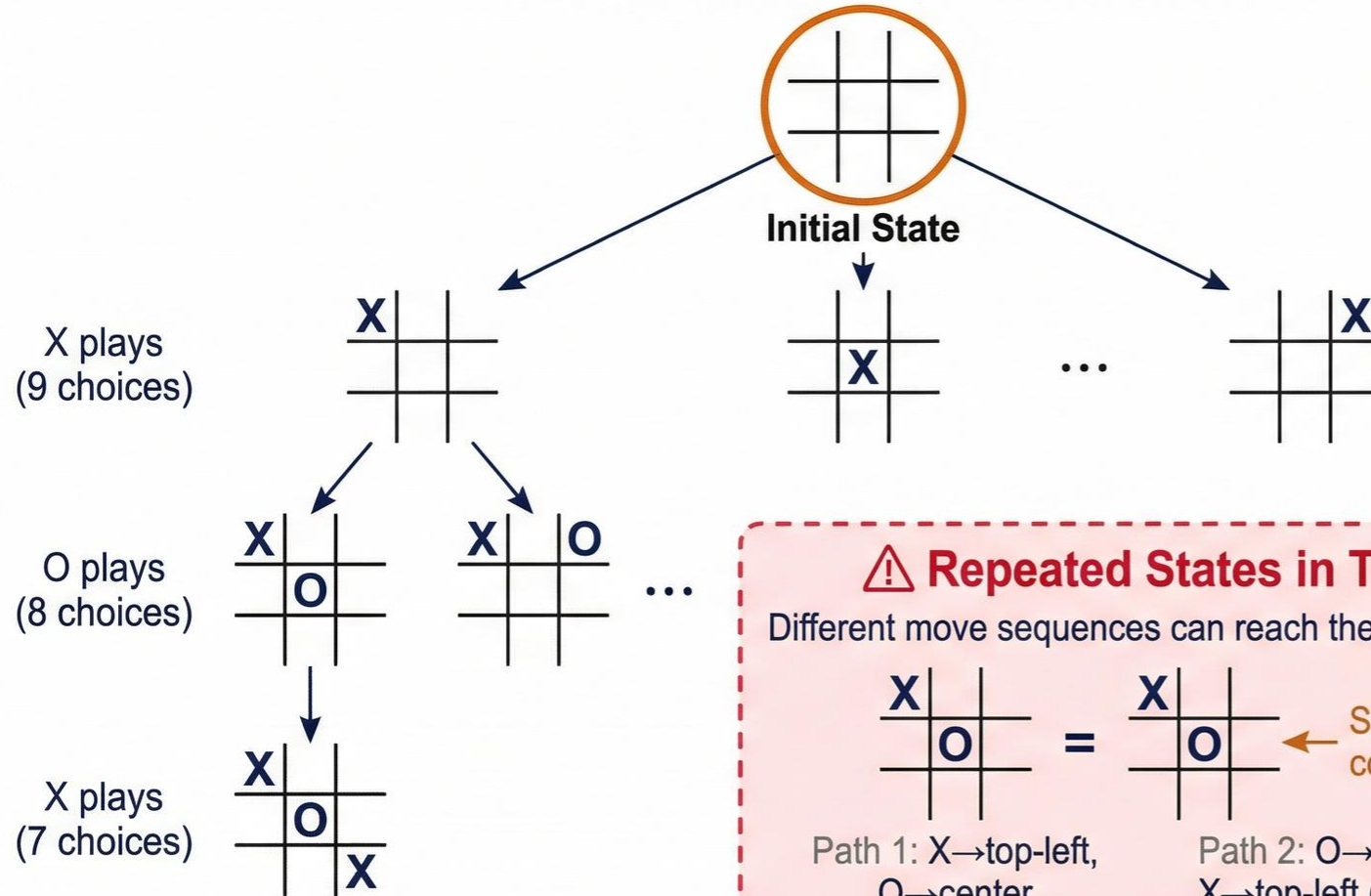
- Oradea: via Zerind (71km) OR via Sibiu (151km)
- Two separate tree nodes, one parent each
- Graph search skips the longer path
- **Each node has exactly ONE parent**

Solution Found at Depth 3

- Arad → Sibiu → Fagaras → Bucharest
- Cost: 140 + 99 + 211 = 450km
- *Is this optimal? (Not necessarily!)*

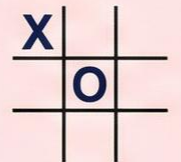
Search Tree as Problem Representation: Tic-Tac-Toe

Partial Search Tree (X moves first)

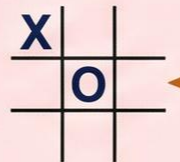


⚠ Repeated States in Tic-Tac-Toe

Different move sequences can reach the SAME board position!



Path 1: X→top-left,
O→center



Path 2: O→center,
X→top-left (different order)

Same state! Wasted
computation if revisited.

Why This Matters

- **State** = board configuration
- **Action** = place X or O
- **Transition** = new board after move
- **Goal test** = three in a row OR board full

Handling Repeated Nodes

Tree Search: revisits states

- May loop forever
- Wastes memory & time

Graph Search: tracks visited

- Uses a “reached” set
- Skips already-seen states

Two Strategies

1. **Check-before-add:**
if state not in reached: add to frontier
2. **Cycle detection:**
check ancestors on path
(lighter but incomplete)

Total tree nodes (naive)

549,946 (includes all paths, early terminations)

After removing repeated states

5,478 unique board positions

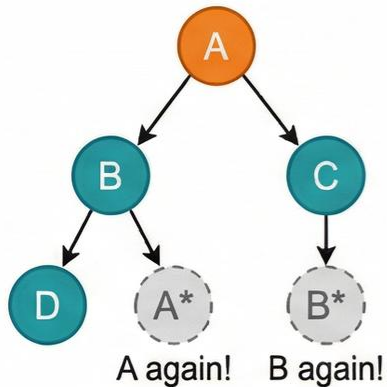
After symmetry reduction

765 canonically distinct positions

Tree Search vs Graph Search

Two strategies for exploring the state space

TREE SEARCH

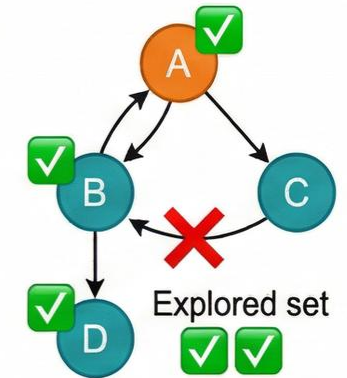


- Explores **paths**, NOT states
- Same state can appear **MULTIPLE** times in different branches
- May revisit states — can loop infinitely
- No memory of visited states
- Simple to implement

Property	Value
Memory usage	Low (no visited set)
Risk of loops	HIGH — can be infinite
Completeness	Not guaranteed
Use when	State space is a tree (no cycles)

⚠ **Problem:** In Romania map, Arad → Sibiu → Arad → Sibiu → ... (infinite loop!)

GRAPH SEARCH



- Explores **states**, NOT just paths
- Maintains an 'explored set' (**closed list**)
- Each state visited **AT MOST ONCE**
- Avoids revisiting states
- More memory required

Property	Value
Memory usage	Higher (stores explored set)
Risk of loops	NONE — states tracked
Completeness	Guaranteed (finite space)
Use when	State space has cycles/repeated states

✓ **Solution:** Add **explored set**. Before expanding a node, check if state was already visited.

Both use a FRONTIER (open list)
Queue of nodes waiting to be explored

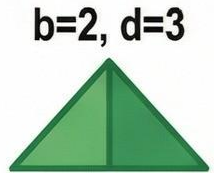
Key Difference
Graph Search adds an **EXPLORED SET**
Tree Search does not

Coming in Session 3
BFS and DFS implement these strategies
Graph search = BFS/DFS + explored set

Why Formulation Matters for Complexity

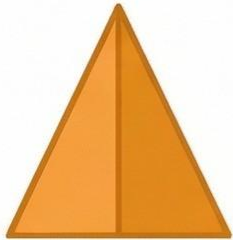
Total Nodes $\approx b^d$

b^d



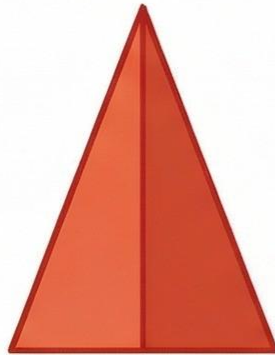
$b=2, d=3$

15 nodes
manageable



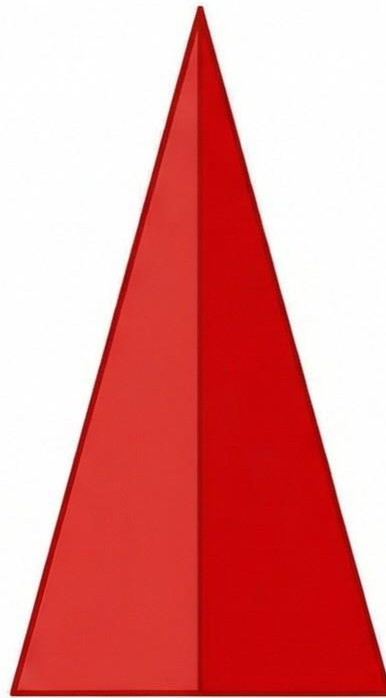
$b=3, d=5$

364 nodes
getting large



$b=5, d=8$

390,625 nodes
challenging



$b=35, d=80$

10^{123} nodes
IMPOSSIBLE

Real Problem Comparison

Problem	b	d	b^d	Feasible?
✓ 8-Puzzle(smart)	~3	~20	~3.5 billion	⚡ With good search
✗ 8-Queens(naive)	64	8	1.8×10^{14}	✗ No
✓ 8-Queens(smart)	~5	8	~390,625	✓ Yes
✓ Water Jug	≤ 9	~6	~531,441	✓ Yes
✗ Chess	~35	~80	10^{123}	✗ Brute force

Smarter Actions $\rightarrow$ Smaller b
This is why formulation matters!
b: 64 $\rightarrow$ 5 = 99.97% reduction

Coming in Session 3-4:
Search algorithms that prune
branches intelligently $\rightarrow A^*$

The Challenge: Combinatorial Explosion

The Numbers Are Staggering

8-Puzzle ($b \approx 3$, $d \approx 20$)

3,486,784,401

≈ 3.5 billion nodes

Still solvable with
smart search

Manageable ✓

Chess ($b \approx 35$, $d \approx 80$)

10^{123}

Shannon Number

More than atoms in universe (10^{80})

Brute force impossible ✗

Go ($b \approx 250$, $d \approx 150$)

10^{360}

Why AlphaGo was revolutionary

Required deep learning + MCTS

Required AI breakthrough ✗

Why This Matters

The Fundamental Challenge of AI Search:

State spaces are almost always
too large to explore completely.

- We need **SMART** search strategies
- Not brute force enumeration

The Solution:

- Heuristics to guide search (Session 4)
- Pruning to eliminate bad branches
- A* algorithm: optimal + efficient
- Deep learning for Go/Chess (AlphaGo)

Today's formulation work is what makes
these smart strategies **POSSIBLE**.

Without a good model, no algorithm can help.

Implementing the Search Tree: make_node & expand

```
Session2_Demo_Problem_Formulation_BFS.ipynb
Session2_Demo_Problem_Formulation_BFS.ipynb x
1
2 def make_node(state, parent=None, action=None, cost=0, depth=0):
3     """Create a search tree node."""
4     return {
5         "state": state,
6         "parent": parent,      # pointer back to parent node
7         "action": action,     # action that led here
8         "cost": cost,         # cumulative path cost
9         "depth": depth        # depth in the tree
10    }
11
12 def expand(problem, node, get_actions_fn, action_cost_fn):
13     """Expand a node: return list of child nodes."""
14     children = []
15     for action_name, next_state in get_actions_fn(problem, node["state"]):
16         step_cost = action_cost_fn(problem, node["state"], action_name)
17         child = make_node(
18             state = next_state,
19             parent = node,
20             action = action_name,
21             cost = node["cost"] + step_cost,
22             depth = node["depth"] + 1
23         )
24         children.append(child)
25     return children
26
27 # Demo: expand the root node
28 root = make_node((0, 0, 0)) # initial state for water jug
29 children = expand(pour_p, root, pour_get_actions, action_cost)
30 print(f"Root has {len(children)} children")
```

Python 3.10 Session 4: expand() Ln 25, Col 1

Node = Snapshot + History

Each node stores:

- state — where we are now
- parent — how we got here
- action — what move was taken
- cost — total path cost so far
- depth — how deep in the tree

expand() = Formulation in Action

expand() calls get_actions_fn() and action_cost_fn() — directly using the 4-component framework.

Good formulation → clean expand() → efficient search.

Why parent pointer?

After reaching the goal, follow parent pointers back to root to reconstruct the solution path.

Arad → Sibiu → Fagaras → Bucharest

PART 5 OF 5

Summary & Next Steps

What we learned · What comes next



Recap



Formulation



Water Jug



Search Tree



Summary

Session 2 Complete

1. 4 Components =
your problem toolkit

2. Actions define b =
search complexity

3. Search tree = agent's
reasoning map

Summary: Key Takeaways

1

Problem Formulation = 4 Components

Initial State · Actions · Goal Test · Path Cost

2

Abstraction is the Core Skill

Keep only what matters for the task

3

Actions Define Branching Factor b

Smarter actions → smaller b → feasible search

4

Search Tree Connects Model to Computation

Nodes=States · Edges=Actions · b^d =Complexity

5

Combinatorial Explosion is Real

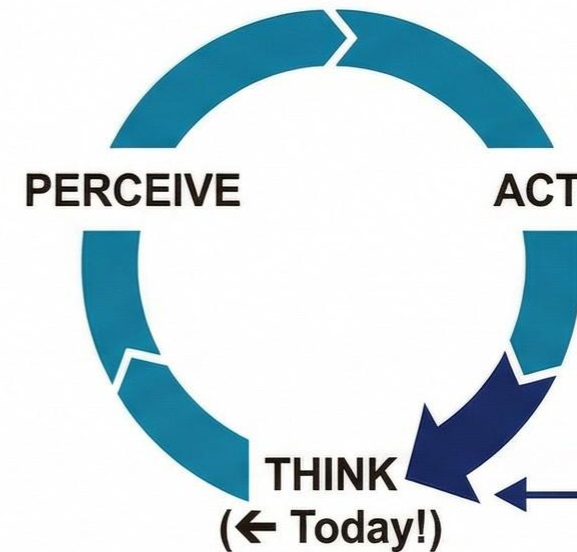
Chess: 10^{123} states — brute force fails

6

Formulation IS the Algorithm

A well-formulated problem is half-solved

The Big Picture



Problem Formulation gives the agent:

- A world model (states)
- A plan space (actions)
- A success criterion (goal)
- A quality measure (cost)



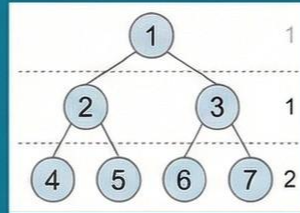
Lab Assignment: Implement WaterJugProblem in Python using the 4-component framework
→ See `Problem_formulation_lecture_demo.ipynb`

Your Next Mission: Teaching the Agent to Search

Session 3 Preview: Uninformed Search

BFS — Breadth-First Search

- Explores level by level
- Guarantees shortest path
- Memory: $O(b^d)$



DFS — Depth-First Search

- Explores one branch fully before backtracking
- Memory efficient: $O(b \times d)$
- May not find shortest path



Session 4 Preview: Informed Search

- A* algorithm uses heuristics to search smarter
- $h(n)$ = estimated cost to goal

Before Next Class

✓ Completed today:

- ✓ Problem formulation framework
- ✓ 4 classic examples
- ✓ Search tree structure
- ✓ Branching factor & complexity

📝 To do before Session 3:

- ☐ Complete Problem_formulation_lecture_demo.ipynb
- ☐ Implement WaterJugProblem class
- ☐ Think: BFS or DFS for Water Jug?

🤔 Think About This:

For the Water Jug problem, would BFS or DFS find the solution faster? Why? Bring your answer to Session 3!

Q&A

Questions & Discussion

Formulation Challenge

Describe a real-world AI problem using the 4-component framework.

(e.g., GPS navigation, robot arm, game AI)

Complexity Calculation

If $b=4$ and $d=10$, how many nodes in the search tree?

$$4^{10} = ?$$

Bonus Question

Can you reduce b for the Water Jug problem?

What actions would you remove?